

DAY 26 | Feature Engineering & Data Preprocessing

60-Day Data Science Learning Journey • Muhammad Ahmad

Learning objectives

Prepare raw data for Machine Learning using missing-value imputation, categorical encoding, scaling, feature engineering, Pipeline, ColumnTransformer, and leakage prevention.

1. Data preprocessing

Data preprocessing transforms raw data into a form suitable for analysis and modeling. Real data may contain missing values, duplicates, inconsistent data types, categories stored as text, outliers, and features on different scales.

2. Feature engineering

Feature engineering creates or transforms features to represent useful information. For example, average order value = total purchase / number of orders. Never use information that would not be available at prediction time.

3. Numerical vs categorical features

Numerical examples: age, salary, study hours, attendance, GPA. Categorical examples: department, city, internet access. Many estimators require categories to be encoded numerically.

4. Missing values

Inspect with `df.isnull().sum()`. `SimpleImputer(strategy="median")` is a common numerical strategy that is less sensitive to outliers than the mean. `SimpleImputer(strategy="most_frequent")` is often used for categorical features. Select a strategy based on the data and why values are missing.

5. Encoding categories

`OneHotEncoder` represents unordered categories using indicator columns. Ordinal encoding can suit genuinely ordered categories, but arbitrary numeric labels for CS, IT, and SE may imply a false order. `handle_unknown="ignore"` helps handle categories not seen during fitting.

6. Feature scaling

`StandardScaler` centers using the training mean and scales by the training standard deviation. `MinMaxScaler` maps values into a chosen range, often 0 to 1. Scaling often matters for distance-based and gradient-based algorithms. Tree models generally need scaling less often.

7. Data leakage

Leakage happens when information unavailable at prediction time influences training or evaluation. Split the data first, fit preprocessing on training data only, and apply the fitted transformations to test data. Pipeline and ColumnTransformer help make this workflow consistent.

8. Pipeline and ColumnTransformer

Pipeline chains transformations and an estimator. ColumnTransformer applies different transformations to different column groups, such as imputation and scaling for numeric columns, and imputation and one-hot encoding for categorical columns.

9. Evaluation

MAE is average absolute error, MSE is average squared error, and R^2 compares a model with a mean-prediction baseline. Interpret metrics together and in context. Tiny synthetic data is for learning, not real academic decisions.

Complete Project | Student Performance ML Pipeline

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = {
    "study_hours": [2,3,4,5,6,7,8,2,4,6,7,9,3,5,8],
    "attendance": [65,70,75,78,82,85,90,None,74,80,88,92,68,76,86],
    "sleep_hours": [5.5,6,6.5,7,7.5,8,5.5,None,7,7.5,8,6,6.5,7.5],
    "previous_gpa": [2.2,2.4,2.5,2.7,2.8,3.0,3.2,2.3,2.6,None,3.1,3.5,2.4,2.7,3.3],
    "department": ["CS","IT","CS","SE","AI","CS","IT","SE","CS","AI","CS","IT","SE","CS","AI"],
    "internet_access": ["Yes","Yes","No","Yes","Yes","Yes","Yes","No","Yes","Yes","No","Yes","No","Yes","Yes"],
    "exam_marks": [42,48,55,62,70,78,86,40,57,72,80,94,50,64,84]
}

df = pd.DataFrame(data)
print("Missing values:", df.isnull().sum())
X = df.drop("exam_marks", axis=1)
y = df["exam_marks"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

numeric_features = ["study_hours", "attendance", "sleep_hours", "previous_gpa"]
categorical_features = ["department", "internet_access"]

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, numeric_features),
    ("categorical", categorical_pipeline, categorical_features)
])
model_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("model", LinearRegression())
])
model_pipeline.fit(X_train, y_train)
y_pred = model_pipeline.predict(X_test)
print("MAE:", round(mean_absolute_error(y_test, y_pred), 2))
print("MSE:", round(mean_squared_error(y_test, y_pred), 2))
print("R2:", round(r2_score(y_test, y_pred), 3))

new_student = pd.DataFrame({
    "study_hours": [6.5], "attendance": [85], "sleep_hours": [7],
    "previous_gpa": [3.1], "department": ["CS"], "internet_access": ["Yes"]
})
print("Predicted marks:", round(model_pipeline.predict(new_student)[0], 2))
```

Line-by-line explanation

Imports

Pandas manages tabular data. Scikit-learn provides splitting, imputers, encoders, scalers, pipelines, regression, and metrics.

DataFrame

`pd.DataFrame(data)` converts the dictionary into rows and columns. None values are interpreted as missing values.

X and y

X contains the input features. y contains exam_marks, the numerical target.

Train/test split

80% of rows train the model and 20% are held out for testing. `random_state=42` makes the split reproducible.

Numeric pipeline

Median imputation fills missing numeric values using a statistic learned from training data. `StandardScaler` then scales those columns.

Categorical pipeline

Most-frequent imputation fills missing categories. OneHotEncoder converts categories into indicator columns and ignores unknown categories during transform.

ColumnTransformer

Applies each pipeline only to its specified columns and combines the transformed outputs.

Model pipeline

Chains preprocessing with LinearRegression so preprocessing is fitted during training and reused for prediction.

Fit and predict

fit learns imputation statistics, category mappings, scaling statistics, and regression parameters from training data. predict applies the learned steps to held-out data.

Metrics

MAE and MSE measure errors; R^2 compares the model with a mean-target baseline. With only 15 synthetic rows, results are highly unstable.

New student

The new DataFrame uses the same feature names. The pipeline preprocesses the input and produces a prediction.

Practice Tasks with Complete Solutions

Task 1: Inspect missing values

```
print(df.isnull().sum())
```

Task 2: Count duplicate rows

```
print(df.duplicated().sum())
```

Task 3: One-hot encode categories

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(handle_unknown="ignore")
encoded = enc.fit_transform([["CS"],["IT"],["SE"],["AI"]])
print(encoded.toarray())
```

Task 4: Numerical preprocessing pipeline

```
numeric_pipeline = Pipeline([("imputer", SimpleImputer(strategy="median")), ("scaler", StandardScaler())])
```

Task 5: Categorical preprocessing pipeline

```
categorical_pipeline = Pipeline([("imputer", SimpleImputer(strategy="most_frequent")), ("encoder", OneHotEncoder(handle_unknown="ignore"))])
```

Task 6: Combine with ColumnTransformer

```
preprocessor = ColumnTransformer([("num", numeric_pipeline, numeric_features), ("cat", categorical_pipeline, categorical_features)])
```

Task 7: Add assignment score

```
df["assignment_score"] = [40,45,55,60,68,75,82,42,58,70,78,95,50,65,85]
# Add assignment_score to numeric_features, then rerun split and training.
```

Task 8: Predict a new student

```
print(model_pipeline.predict(new_student))
```

Task 9: Explain leakage prevention

Split first. Fit preprocessing only through `model_pipeline.fit(X_train, y_train)`. Evaluate with `model_pipeline.predict(X_test)`. Do not fit imputer or encoder on test data.

Task 10: Compare scaling methods

```
from sklearn.preprocessing import MinMaxScaler
minmax = MinMaxScaler()
# In a pipeline, replace StandardScaler() with MinMaxScaler() and compare held-out metrics.
```

Day 26 quick revision

Preprocessing: clean and transform raw data.

Feature engineering: create useful representations.

Imputation: handle missing values.

One-Hot Encoding: represent unordered categories.

StandardScaler: center and scale numeric features.

Pipeline: chain preprocessing and model.

ColumnTransformer: different transformations for different columns.

Leakage prevention: fit preprocessing on training data only.

Important note

This tiny synthetic dataset is for learning only and must not be used for real academic decisions. Real systems require representative data, validation, leakage checks, appropriate feature selection, and domain review.